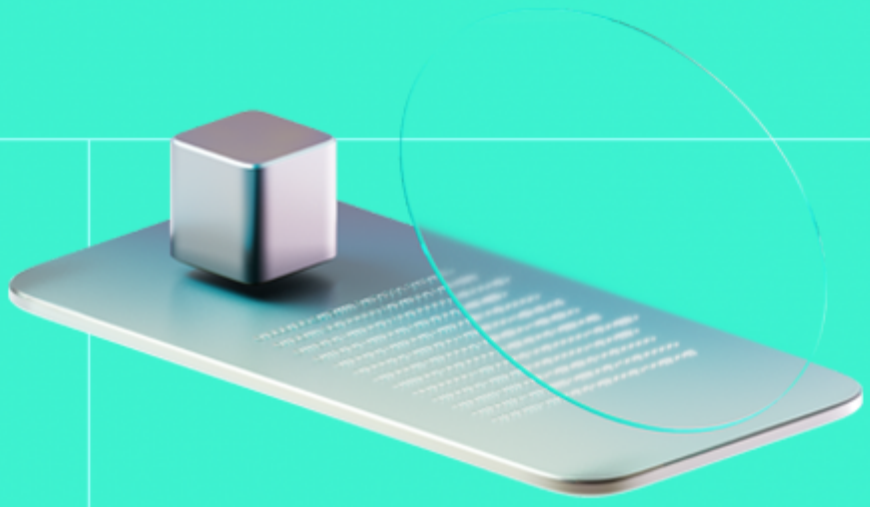
HACKEN

Smart Contract Code Review And Security Analysis Report



Customer: EverValue Coin

Date: 11/02/2025

We express our gratitude to the EverValue Coin team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

EverValue Coin (EVA) is a new token with real value in bitcoins because it is fully backed by Bitcoins. It has a unique and final issuance of 21 million, making it better than Bitcoin in this aspect. It operates in a "smart contract" wallet where new bitcoins are deposited daily but can only be withdrawn by burning EVA tokens.

Document

Name	Smart Contract Code Review and Security Analysis Report for EverValue Coin
Audited By	Seher Saylik, Andy Cho
Approved By	Ataberk Yavuzer
Website	https://evervaluecoin.com/
Changelog	10/12/2024 - Preliminary Report
Changelog	11/02/2025 - Final Report
Platform	Arbitrum network
Language	Solidity
Tags	OrderBook, ERC-20, DEX
Methodology	https://hackenio.cc/sc_methodology

Review Scope

Repository	https://github.com/devervalue/orderbook
Commit	c591236

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

5	4	1	0
Total Findings	Resolved	Accepted	Mitigated

During the audit, five issues were identified: two high, two medium, and one low. The **development team promptly addressed and resolved two high, one medium, and one low-severity issue, which were subsequently retested and approved in the final review. One medium-severity issue is acknowledged,** as the team confirmed they will not use fee-on-transfer functionality.

This report reflects the final state of the code after all necessary fixes and improvements were implemented.

Findings by Severity

Severity	Count
Critical	0
High	2
Medium	2
Low	1

Vulnerability	Severity
F-2024-7557 - Stuck Order Matching Due to Blacklisted Addresses	High
F-2025-8721 - Exploitable Order Quantity Leading to Fund Loss	High
F-2024-7553 - Vulnerability to Fee-on-Transfer Accounting Issues	Medium
F-2025-8551 - Rounding Issue in fillOrder and partiallyFillOrder Allows Free Token Transfer	Medium
F-2024-7555 - Use Ownable2Step Rather Than Ownable	Low

Documentation quality

- Functional requirements are provided.
- Technical description is provided.

Code quality

- The development environment is configured.

Test coverage

Code coverage of the project is **100%** (branch coverage).

- Deployment and basic user interactions are covered with tests.
- Negative cases coverage is covered.
- Interactions by several users are tested thoroughly.

Table of Contents

System Overview

The **OrderBook** Smart Contract is an on-chain decentralized exchange (DEX) implementation that aims to provide a fully transparent and efficient trading platform for ERC20 tokens. Unlike traditional Automated Market Maker (AMM) based DEXes, this system implements a limit order book model, similar to those used in centralized exchanges.

OrderBookFactory — main orderbook contract that manages the creation and administration of order books for trading pairs, allowing users to place and manage buy/sell orders efficiently. It supports features like adding new pairs, updating fees, pausing operations.

OrderBookLib — A library for managing an order book.

PairLib — A library designed for managing trading pairs and their associated order books in decentralized exchanges. It supports creating, canceling, and matching buy/sell orders while handling functionalities such as maintaining trader-specific registries, tracking order states, and calculating trading fees.

QueueLib — A library that implements a queue data structure.

RedBlackTreeLib — A library that implements a Red-Black Tree data structure.

Privileged roles

- The owner role can add new pairs, change the enabled status for a trading pair, update the fee for a specific trading pair, set a new fee recipient address for a specific order book, and pause/unpause the contract.

Potential Risks

- **Insufficient Multi-signature Controls for Critical Functions:** The lack of multi-signature requirements for key operations centralizes decision-making power, increasing vulnerability to single points of failure or malicious insider actions, potentially leading to unauthorized transactions or configuration changes.
- The reliance on off-chain price management and user responsibility for accurate decimals handling introduces a potential risk of incorrect price submissions, which might result in token mispricing. Without on-chain validation or adjustment for token decimals, the system remains susceptible to errors or exploitation if backend systems or user inputs are compromised.
- The `matchOrder` function's loop, while capped at 1500 iterations, poses a potential DoS risk as processing large orders involving numerous matching transactions could consume excessive gas. This may prevent the completion of order processing, disrupting contract operations for legitimate users.

Findings

Vulnerability Details

F-2024-7557 - Stuck Order Matching Due to Blacklisted Addresses - High

Description:

The `OrderBookFactory` contract faces an issue when matching buy or sell orders. If an order involves a creator's address that has been blacklisted by the token contract (e.g., a token implementing blacklist functionality), the transfer of funds between the buyer and seller will fail. This failure prevents the system from proceeding to the next price point, effectively causing the matching process to halt indefinitely at the problematic price level.

Stalled Order Matching:

- The matching process cannot move past the blacklisted order, blocking all subsequent orders at that price point or higher/lower (depending on buy/sell direction).
- This creates a significant bottleneck in the order book, reducing its efficiency and usability.

Permanent Price Point Blockage:

- Price points with blacklisted addresses become unusable, causing funds to remain locked and orders to stay unresolved indefinitely.

Affected functions:

```
        function fillOrder(Pair storage pair, OrderBookLib.Order storage matc
hedOrder, OrderBookLib.Order memory takerOrder)
        private
    {
        // Update the last trade price for the pair
        pair.lastTradePrice = matchedOrder.price;

        // Determine which tokens are being received and sent by the taker, a
nd their amounts
        (IERC20 takerReceiveToken, uint256 takerReceiveAmount, IERC20 takerSe
ndToken, uint256 takerSendAmount) =
        takerOrder.isBuy
            ? (
                IERC20(pair.baseToken),
```

```
                matchedOrder.availableQuantity,
                IERC20(pair.quoteToken),
                matchedOrder.availableQuantity * matchedOrder.price / PRECISI
ON
            )
            : (
                IERC20(pair.quoteToken),
                matchedOrder.availableQuantity * matchedOrder.price / PRECISI
ON,
                IERC20(pair.baseToken),
                matchedOrder.availableQuantity
            );

        // Calculate the fee based on the amount the taker receives
        /// @dev The fee is calculated in basis points (1/100 of a percent)
        uint256 fee = (takerReceiveAmount * pair.fee) / 10000;
        uint256 takerReceiveAmountAfterFee = takerReceiveAmount - fee;

        // Transfer tokens between the taker and the maker
        /// @dev The taker sends the full amount, while receiving the amount
minus the fee
        takerSendToken.safeTransferFrom(msg.sender, matchedOrder.traderAddres
s, takerSendAmount);
        takerReceiveToken.safeTransfer(msg.sender, takerReceiveAmountAfterFee
);

        // Transfer the fee to the designated fee address, if set
        if (pair.feeAddress != address(0)) {
            takerReceiveToken.safeTransfer(pair.feeAddress, fee);
        }

        // Update the taker's order quantity
        takerOrder.quantity -= matchedOrder.availableQuantity;

        // Emit events for the filled orders
        emit OrderFilled(matchedOrder.id, pair.baseToken, pair.quoteToken, ma
tchedOrder.traderAddress);

        // Remove the fully matched order from the order book
        removeOrder(pair, matchedOrder);
    }
```

```
    function partiallyFillOrder(
        Pair storage pair,
        OrderBookLib.Order storage matchedOrder,
        OrderBookLib.Order memory takerOrder
    ) private {
        // Update the last trade price for the pair
```

```
        pair.lastTradePrice = matchedOrder.price;
        // Determine which tokens are being received and sent by the taker, an
d their amounts
        /// @dev The calculation depends on whether the taker order is a buy o
r sell order
        (IERC20 takerReceiveToken, uint256 takerReceiveAmount, IERC20 takerSen
dToken, uint256 takerSendAmount) =
        takerOrder.isBuy
            ? (
                IERC20(pair.baseToken),
                takerOrder.quantity,
                IERC20(pair.quoteToken),
                takerOrder.quantity * matchedOrder.price / PRECISION
            )
            : (
                IERC20(pair.quoteToken),
                takerOrder.quantity * matchedOrder.price / PRECISION,
                IERC20(pair.baseToken),
                takerOrder.quantity
            );
        // Calculate fee (on the buy token amount, which is what the taker rec
eives)
        /// @dev The fee is calculated in basis points (1/100 of a percent)
        uint256 fee = (takerReceiveAmount * pair.fee) / 10000;
        uint256 takerReceiveAmountAfterFee = takerReceiveAmount - fee;
        // Transfer sell tokens from taker to maker (full amount)
        takerSendToken.safeTransferFrom(msg.sender, matchedOrder.traderAddress
, takerSendAmount);
        // Transfer buy tokens from maker to taker (minus fee)
        takerReceiveToken.safeTransfer(msg.sender, takerReceiveAmountAfterFee)
;
        // Transfer fee to fee address if set, otherwise it stays in the contr
act
        if (pair.feeAddress != address(0)) {
            takerReceiveToken.safeTransfer(pair.feeAddress, fee);
        }
        // Update the matched order by reducing its available quantity
        matchedOrder.availableQuantity = matchedOrder.availableQuantity - take
rOrder.quantity;
        matchedOrder.status = ORDER_PARTIALLY_FILLED;
        // Update the order book to reflect the partial fill
        /// @dev This updates the volume at the price point in the order book
        (matchedOrder.isBuy ? pair.buyOrders : pair.sellOrders).update(matched
Order.price, takerOrder.quantity);
        // Set the taker order quantity to 0 as it has been fully filled
        takerOrder.quantity = 0;
        // Emit an event for the filled taker order
        emit OrderFilled(takerOrder.id, pair.baseToken, pair.quoteToken, msg.s
ender);
```

```
        // Emit an event for the partially filled matched order
        emit OrderPartiallyFilled(matchedOrder.id, pair.baseToken, pair.quoteT
oken, matchedOrder.traderAddress);
    }
```

The following line in the `fillOrder()` function is vulnerable to blacklisted transfers:

```
/// @dev The taker sends the full amount, while receiving the amount minus th
e fee
takerSendToken.safeTransferFrom(msg.sender, matchedOrder.traderAddress, taker
SendAmount);
```

The following line in the `partiallyFillOrder()` function is vulnerable to blacklisted transfers:

```
// Transfer sell tokens from taker to maker (full amount)
takerSendToken.safeTransferFrom(msg.sender, matchedOrder.traderAddress, taker
SendAmount);
```

Assets:

- OrderBookFactory.sol [https://github.com/devervalue/orderbook]

Status: Fixed

Classification

Impact: 4/5

Likelihood: 4/5

Exploitability: Independent

Complexity: Simple

Severity: High

Recommendations

Remediation: To address this issue, implement a fallback mechanism to handle failed transfers during the matching process and allow the system to skip problematic orders. Additionally, introduce a recovery mechanism for funds associated with such orders.

Resolution: The EverValue Coin team implemented a pull-over-push pattern to prevent stuck orders that disrupt the orderbook mechanism.

(Revised commit: a958f6e)

Evidences

PoC

Reproduce:

. Steps to reproduce the issue;

Test Setup:

- **Contracts Deployed**: `OrderBookFactory`, `MyTokenA`, and `SimpleERC20` with initial supplies.
- **Tokens Transferred**: Traders receive tokens and approve the factory contract for token spending.

Placing Buy Orders

- **Trader1** places a buy order for `tokenB` at a price of 100 and an amount of 1 token.

Blacklisting and Failed Sell Order

- After the buy order is placed, **Trader1** is blacklisted by the `tokenB` contract.
- **Trader2** attempts to match the buy order by placing a sell order at the same price and amount.
- The sell order fails because `tokenB` blocks transfers to blacklisted addresses.

The buy order from **Trader1** is stuck in the order book, blocking all future trades at the price point of 100.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;
import "forge-std/Test.sol";
import "../src/OrderBookFactory.sol";
import "./MyTokenB.sol";
import "./MyTokenA.sol";
import "forge-std/console.sol";
import "../src/MockTokenBlacklist.sol";
contract OrderBookFactoryTest is Test {
   OrderBookFactory factory;
   // ADDRESSES
   address owner = makeAddr("owner");
   address feeAddress = makeAddr("feeAddress");
   address trader1 = makeAddr("trader1");
   address trader2 = makeAddr("trader2");
   address trader3 = makeAddr("trader3");
   // TOKENS
```

```
    MyTokenA tokenA;
    SimpleERC20 tokenB;

    function setUp() public {
        vm.prank(owner);
        factory = new OrderBookFactory();
        // Creating tokens with an initial supply
        tokenA = new MyTokenA(1000 * 10 ** 18); // Create a new token with ini
tial supply
        tokenB = new SimpleERC20("Test Token", "Te
```

See more

F-2025-8721 - Exploitable Order Quantity Leading to Fund Loss - High

Description:

A vulnerability in the order book logic allows an attacker to exploit mismatches in order fulfillment, leading to fund theft or permanent fund freezing. The core issue arises because the `availableQuantity` of a completely fulfilled or a partially fulfilled order is not updated, leading to inconsistencies in order cancellation and fulfillment by other takers.

When an order is partially or fully matched, only the `quantity` field is updated, while the `availableQuantity` remains unchanged. This discrepancy allows an attacker to manipulate the system by either canceling the order and reclaiming excess funds or having another trader fulfill the order again for an amount greater than what remains.

The vulnerability originates from the `matchOrder()` function:

```
if (newOrder.quantity >= matchingOrder.availableQuantity) {
   fillOrder(pair, matchingOrder, newOrder);
} else {
   partiallyFillOrder(pair, matchingOrder, newOrder);
   return (newOrder.quantity, orderCount);
}
```

The `fillOrder` function correctly deducts `quantity`, but `availableQuantity` is not updated:

```
takerOrder.quantity -= matchedOrder.availableQuantity;
```

This issue persists when the remaining order is stored back into the queue:

```
if (_quantity > 0) {
   addOrder(pair, newOrder);
}
```

Since `availableQuantity` was not updated, it remains at its original value, allowing multiple claims over the same capital.

The same missing state update of `availableQuantity` is also present in the `partiallyFillOrder()` function.

An attacker can exploit this issue to withdraw more funds than they originally deposited if there is pending orders in the system and it is

cancelled at the correct moment.

Assets:

- PairLib.sol [https://github.com/devervalue/orderbook]

Status: Fixed

Classification

Impact: 5/5

Likelihood: 4/5

Exploitability: Independent

Complexity: Medium

Severity: High

Recommendations

Remediation: Update `availableQuantity` when an order is fulfilled or partially filled. This ensures that future interactions (cancellations or additional matches) use the correct amount, preventing fund duplication and insolvency risks.

Resolution: The EverValue Coin team fixed the issue by updating the `availableQuantity` in order filling functions.
(Revised commit: 1516471)

Evidences

PoC

Reproduce:

Steps to reproduce the issue

User A places a buy order for 100,000 USDC.

User B places a sell order for 200,000 USDC, fulfilling User A’s order.

User B’s remaining 100,000 USDC is stored back into the order book but with an incorrect `availableQuantity` of 200,000.

User C submits a matching buy order for 200,000 USDC, fulfilling User B’s remaining order.

Result**:** User B gets paid twice for the same 100,000 USDC, effectively stealing funds.

Results:

```
//For simplicity purposes token ratio(TokenA/TokenB) is 1:1 (price of 1e18)
   function testAvailableQuantityExploit() public {
        vm.prank(owner);
        factory.addPair(address(tokenA), address(tokenB), 5, feeAddress);
        //Balance of both traders /both tokens prior to exploit
        uint256 tokenAtrader1 = tokenA.balanceOf(trader1);
        uint256 tokenBtrader1 = tokenB.balanceOf(trader1);
        //Both traders have 250k balances on both tokens;
        console.log("Token A trader1 balance before: ", tokenAtrader1 / 1e18);
        console.log("Token B trader1 balance before: ", tokenBtrader1 / 1e18);
        uint256 tokenAtrader2 = tokenA.balanceOf(trader2);
        uint256 tokenBtrader2 = tokenB.balanceOf(trader2);
        console.log("Token A trader2 balance before: ", tokenAtrader2 / 1e18);
        console.log("Token B trader2 balance before: ", tokenBtrader2 / 1e18);
        uint256 price = 1e18;
        bytes32[] memory keys = factory.getPairIds();
        //Trader1 is the victim, they place a buy order for a quantity of 10_0
00e18 tokens at a price of 1e18;
        vm.prank(trader1);
        factory.addNewOrder(keys[0], 10_000e18, price, true, 1);
        //New blanace of the quote token of trade 1
        //Here trader1 has a 240k balance on tokenA (10k was transferred to fa
ctory);
        uint256 tokenAtrader1afterOrder = tokenA.balanceOf(trader1);
        console.log("Token A trader1 balance after order posted: ", tokenAtrad
er1afterOrder / 1e18);


        //Trader2 is the perpetrator, they place an order for 20_000e18, out o
f which 10_000e18 will be covered immediately from trader1's order and the ot
her 10_000e18 will be part of the new order.
        vm.prank(trader2);
        factory.addNewOrder(keys[0], 20_000e18, price, false, 1);
        vm.prank(trader1);
        factory.withdrawBalanceTrader(keys[0], true);
        //Balances of both traders after trader1's order is fully executed, ba
lance withdrawn and trader 2's or
```

See more

F-2024-7553 - Vulnerability to Fee-on-Transfer Accounting Issues - Medium

Description:

The contract uses `transferFrom()` in multiple instances without verifying whether the actual tokens received match the intended transfer amount. If the tokens involved are fee-on-transfer tokens, a portion of the transferred tokens may be deducted as fees, leading to discrepancies in accounting. This can result in unexpected behavior, including allowing malicious actors to exploit latent balances to gain free credits or bypass intended restrictions.

In fee-on-transfer tokens, a percentage of the transferred amount is deducted as a fee during every transfer. This deduction can result in the recipient receiving fewer tokens than specified in the transfer function. The following functions in the contract are vulnerable to this issue:

Affected code parts:

./src/PairLib.sol

```
219: token.safeTransferFrom(msg.sender, address(this), transferAmount);
280: takerSendToken.safeTransferFrom(msg.sender, matchedOrder.traderAddress,
takerSendAmount);
334: takerSendToken.safeTransferFrom(msg.sender, matchedOrder.traderAddress,
takerSendAmount);
```

Impact

Incorrect Accounting: The recipient balance will be lower than expected, potentially leading to failed transactions or mismatches in logic.

Assets:

- PairLib.sol [https://github.com/devervalue/orderbook]

Status: Accepted

Classification

Impact: 4/5

Likelihood: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: To mitigate potential vulnerabilities and ensure accurate accounting with fee-on-transfer tokens, modify your contract's token transfer logic to measure the recipient's balance before and after the transfer. Use this observed difference as the actual transferred amount for any further logic or calculations. For example:

```
function transferTokenAndPerformAction(address token, address from, address t
o, uint256 amount) public {
    uint256 balanceBefore = IERC20(token).balanceOf(to);

    // Perform the token transfer
    IERC20(token).transferFrom(from, to, amount);

    uint256 balanceAfter = IERC20(token).balanceOf(to);
    uint256 actualReceived = balanceAfter - balanceBefore;

    // Proceed with logic using actualReceived instead of the initial amount
    require(actualReceived >= minimumRequiredAmount, "Received amount is less
than required");

    // Further logic here
}
```

Resolution: The EverValue team accepted the risk for the giving finding by stating that;

> In relation to the vulnerability identied in the contract, it has been determined that, for the time being, ERC20 tokens with fee-on-transfer will not be supported. This is due to the fact that tokens with this feature deduct a percentage of the transferred amount as a fee during each transfer, which can cause discrepancies in the amount received by the recipient compared to what was expected. This behavior creates risks of balance inconsistency and could potentially be exploited by malicious actors to gain free credits or bypass intended restrictions in the contract.
> To mitigate this vulnerability, the use of tokens with this functionality will be disabled until an appropriate handling of transfer fees is implemented.

F-2025-8551 - Rounding Issue in fillOrder and partiallyFillOrder Allows Free Token Transfer - Medium

Description:

The `fillOrder()` and `partiallyFillOrder()` functions in `PairLib.sol` suffer from an **integer division rounding issue** when calculating `takerSendAmount` (the amount of quote tokens the taker needs to transfer).

Solidity uses **integer division**, meaning that when performing the calculation:

```
takerSendAmount = matchedOrder.availableQuantity * matchedOrder.price / PRECISION;
```

If `(matchedOrder.availableQuantity * matchedOrder.price)` is **less than** `PRECISION`, the result will be truncated to **zero** due to integer division, **allowing the order to be executed without transferring any quote tokens**.

This enables an attacker to **receive base tokens without paying for them** by deliberately placing orders at small quantities and low prices where rounding errors occur.

Assets:

- PairLib.sol [https://github.com/devervalue/orderbook]

Status: Fixed

Classification

Impact: 2/5

Likelihood: 4/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: To mitigate the rounding issue, introduce a validation check in the `fillOrder` and `partiallyFillOrder` functions. Ensure that `takerSendAmount` and `takerReceiveAmount` are greater than zero before proceeding with

token transfers. Additionally, the `addOrder` function should revert if the transfer amount is zero. For partial filling, the order filling should be skipped if the transferred amount is zero, and the taker's order should be finalized.

Resolution: The EverValue Coin team introduced necessary checks to fix the issue. When `takerSendAmount` or `takerReceiveAmount` is zero, the `fillOrder()` function now reverts, and the `partiallyFillOrder()` function skips order filling and finalizes the taker's order. Lastly, the `addOrder()` function reverts when `takerSendAmount` is zero (Revised commit: ef39ea0).

F-2024-7555 - Use Ownable2Step Rather Than Ownable - Low

Description: The protocol implements single step ownership transfer pattern. Thus, whenever the new owner is set incorrectly, the access to all protected functionality by the `onlyOwner` modifier is permanently lost.

```
contract OrderBookFactory is ReentrancyGuard, Pausable, Ownable {
```

Assets:

- OrderBookFactory.sol [https://github.com/devervalue/orderbook]

Status: Fixed

Classification

Impact: 5/5

Likelihood: 1/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Consider using **Ownable2Step** or **Ownable2StepUpgradeable** from OpenZeppelin Contracts to enhance the security of your contract ownership management. These contracts prevent the accidental transfer of ownership to an address that cannot handle it, such as due to a typo, by requiring the recipient of owner permissions to actively accept ownership via a contract call. This two-step ownership transfer process adds an additional layer of security to your contract's ownership management.

Resolution: The EverValue Coin team introduced Ownable2Step for ownership management. **(Revised commit: 32c2ae2)**

Observation Details

F-2024-7531 - Remove `console` import - Info

Description: In Solidity development, `foundry` is a smart contract development toolchain. However, `console` import in your Solidity code, such as those used for testing or local development, should not make their way into the final production version of the contract. These imports can bloat the contract, lead to unnecessary complexity, and potentially introduce security risks or unexpected behavior. Ensuring that production code is clean, efficient, and free of development-only dependencies is crucial for maintaining security and performance.

```
import "forge-std/console.sol";
```

Assets:

- PairLib.sol [https://github.com/devervalue/orderbook]
- OrderBookLib.sol [https://github.com/devervalue/orderbook]
- QueueLib.sol [https://github.com/devervalue/orderbook]
- RedBlackTreeLib.sol [https://github.com/devervalue/orderbook]

Status: Fixed

Recommendations

Remediation: Before finalizing and deploying your Solidity contracts, thoroughly review the code to ensure all `console` imports and related development-only code are removed.

Resolution: The EverValue Coin removed the "console"' import. **(Revised commit: 32c2ae2)**

F-2024-7535 - Floating pragma - Info

Description: The project uses a floating Pragma.
This may result in the contracts being deployed using the wrong Pragma version, which is different from the one they were tested with. For example, they might be deployed using an outdated Pragma version, which may include bugs that affect the system negatively.

```
pragma solidity ^0.8.26;
```

Assets:

- RedBlackTreeLib.sol [https://github.com/devervalue/orderbook]
- interface/IOrderBookFactory.sol [https://github.com/devervalue/orderbook]
- QueueLib.sol [https://github.com/devervalue/orderbook]
- OrderBookLib.sol [https://github.com/devervalue/orderbook]
- PairLib.sol [https://github.com/devervalue/orderbook]
- OrderBookFactory.sol [https://github.com/devervalue/orderbook]

Status: Fixed

Recommendations

Remediation: Consider locking the Pragma version whenever possible and avoid using a floating Pragma in the final deployment. Consider known bugs for the compiler version that is chosen.

Resolution: The floating pragma is removed from the contracts. **(Revised commit: 32c2ae2)**

F-2024-7554 - Redundant Quantity Check in addNewOrder Function - Info

Description:

The `addNewOrder()` function in the `OrderBookFactory` contract contains a redundant check for the validity of the order quantity (`_quantity`). This check is unnecessary because it is already performed in the respective functions within the `PairLib` library (`addBuyOrder()` and `addSellOrder()`).

Affected code parts:

```
    function addNewOrder(bytes32 _pairId, uint256 _quantity, uint256 _price,
bool _isBuy, uint256 _timestamp)
        external
        onlyEnabledPair(_pairId)
        nonReentrant
        whenNotPaused
    {
        if (_quantity == 0) revert OBF__InvalidQuantityValueZero();
        if (_isBuy) {
            pairs[_pairId].addBuyOrder(_price, _quantity, _timestamp);
        } else {
            pairs[_pairId].addSellOrder(_price, _quantity, _timestamp);
        }
    }
```

The redundant check increases the gas cost of the `addNewOrder()` function.

Assets:

- OrderBookFactory.sol [https://github.com/devervalue/orderbook]

Status: Fixed

Recommendations

Remediation: Remove the redundant check from the `addNewOrder()` function in the `OrderBookFactory` contract. The validation is already handled in the `PairLib` library, ensuring that invalid quantities are rejected.

Resolution: The redundant check for order quantity (`_quantity`) in the `addNewOrder` function of the `OrderBookFactory` contract has been removed by EverValue Coin team. **(Revised commit: 32c2ae2)**

F-2024-7556 - Violation of Checks-Effects-Interactions (CEI) Pattern in cancelOrder() - Info

Description:

The `cancelOrder()` function in the `PairLib` library violates the Checks-Effects-Interactions (CEI) pattern by performing an external token transfer (`token.safeTransfer`) before updating the internal state of the contract.

The following code block from the `cancelOrder` function transfers tokens before the internal state is updated:

```
// Transfer the remaining funds back to the trader
token.safeTransfer(removedOrder.traderAddress, remainingFunds);
// Remove the order from the order book and related data structures
removeOrder(pair, removedOrder);
```

Impact

Reentrancy Risk:

- If the `safeTransfer` function triggers a malicious contract with a reentrant call, the internal state of the order book (`pair.orders` and related mappings) could be manipulated, leading to potential exploits such as double withdrawals or inconsistencies.

State Inconsistency:

- In the event of a failed token transfer, the state of the contract remains unmodified, leading to a mismatch between the actual state of the contract and the intended changes.

Assets:

- PairLib.sol [https://github.com/devervalue/orderbook]

Status: Fixed

Recommendations

Remediation:

Adhere to the Checks-Effects-Interactions (CEI) pattern by updating the internal state of the contract before performing any external interactions. Specifically: Call `removeOrder()` to update the internal state before executing the token transfer.

Resolution:

The `cancelOrder()` function in the `PairLib` library is updated to follow the Checks-Effects-Interactions (CEI) pattern by the EverValue Coin

team. **(Revised commit: 32c2ae2)**

F-2024-7559 - Shortcircuit Rules Can Be Used To Optimize Some Gas Usage - Info

Description:

Some conditions may be reordered to save an SLOAD (2100 gas), as we avoid reading state variables when the first part of the condition fails (with **&&**), or succeeds (with **||**).

Affected code parts:

./src/RedBlackTreeLib.sol

```
223: while (_cursor != EMPTY && value == self.nodes[_cursor].right) {
260: while (_cursor != EMPTY && value == self.nodes[_cursor].left) {
```

Assets:

- RedBlackTreeLib.sol [https://github.com/devervalue/orderbook]

Status: Accepted

Recommendations

Remediation:

Review your Solidity contracts for conditional statements that use **&&** and **||** operators. Prioritize conditions that are less gas-intensive or have a higher likelihood of determining the outcome of the entire expression. For instance, if a condition involves an **SLOAD** operation and another is a simple variable comparison, evaluate the simpler condition first: Before optimization:

```
if (contractStateVariable > value && isEligible(msg.sender)) {
    // Execution logic
}

After optimization:
if (isEligible(msg.sender) && contractStateVariable > value) {
    // Execution logic
}

This rearrangement ensures that if `isEligible(msg.sender)` returns false, th
e contract avoids the gas cost associated with reading `contractStateVariable
` from storage. Apply this principle across your contract's logic where appli
cable, especially in functions called frequently or in gas-sensitive contexts
. Testing remains crucial; ensure that the reordering of conditions does not
alter the intended logic or outcomes of your contract.
```

Resolution: The EverValue Coin stated that

> All relevant conditions in the contract have been reviewed, and it has been determined
> that the current implementation is already optimized for gas usage.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

hknio/severity-formula

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project’s performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/devervalue/orderbook
Commit	c591236aab86b61df2d23630b4b9d192db65a606
Retest commit	151647111838bf904e2a9a27f8afc5575f9a2cc0
Whitepaper	https://evervaluecoin.com/white-paper/
Requirements	https://github.com/devervalue/orderbook/blob/master/README.md
Technical Requirements	https://github.com/devervalue/orderbook/blob/master/README.md
Deployed contracts	OrderBookFactory - 0x03339ECAE41bc162DAcAe5c2A275C8f64D6c80A0 (Arbirtum Chain)

Asset	Type
interface/IOrderBookFactory.sol [https://github.com/devervalue/orderbook]	Smart Contract
OrderBookFactory.sol [https://github.com/devervalue/orderbook]	Smart Contract
OrderBookLib.sol [https://github.com/devervalue/orderbook]	Smart Contract
PairLib.sol [https://github.com/devervalue/orderbook]	Smart Contract
QueueLib.sol [https://github.com/devervalue/orderbook]	Smart Contract
RedBlackTreeLib.sol [https://github.com/devervalue/orderbook]	Smart Contract

Appendix 3. Additional Valuables

Verification of System Invariants

During the audit of EverValue Coin, Hacken followed its methodology by performing fuzz-testing on the project's main functions. Foundry, a tool used for fuzz-testing, was employed to check how the protocol behaves under various inputs. Due to the complex and dynamic interactions within the protocol, unexpected edge cases might arise. Therefore, it was important to use fuzz-testing to ensure that several system invariants hold true in all situations.

Fuzz-testing allows the input of many random data points into the system, helping to identify issues that regular testing might miss. A specific, 11 invariants were tested over 50,000 runs. This thorough testing ensured that the system works correctly even with unexpected or unusual inputs.

Invariant	Test Result	Run Count
`RedBlackTreeLib::insert`. `RedBlackTreeLib::remove` should insert and remove consistenly	Passed	50k
`RedBlackTreeLib::remove` multiple random removal should adjust the first price correctly	Passed	50k
`RedBlackTreeLib::next` should bring the correct next price	Passed	50k
`RedBlackTreeLib::next` should bring the correct prev price	Passed	50k
`PairLib::addBuyOrder` add any orders successfully	Passed	50k
`PairLib::cancelOrder` remove any orders successfully	Passed	50k
`OrderBookFactory::addPair` add pairs successfully	Passed	100k
`OrderBookFactory::setPairStatus` set the pair status successfully	Passed	100k
`OrderBookFactory::setPairFee` set the pair fee successfully	Passed	100k
`OrderBookFactory::setPairFeeAddress` set the pair fee address successfully	Passed	100k
`OrderBookFactory::addNewOrder` add new orders successfully	Passed	100k

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.